Section 13

Section 12

Image Processing

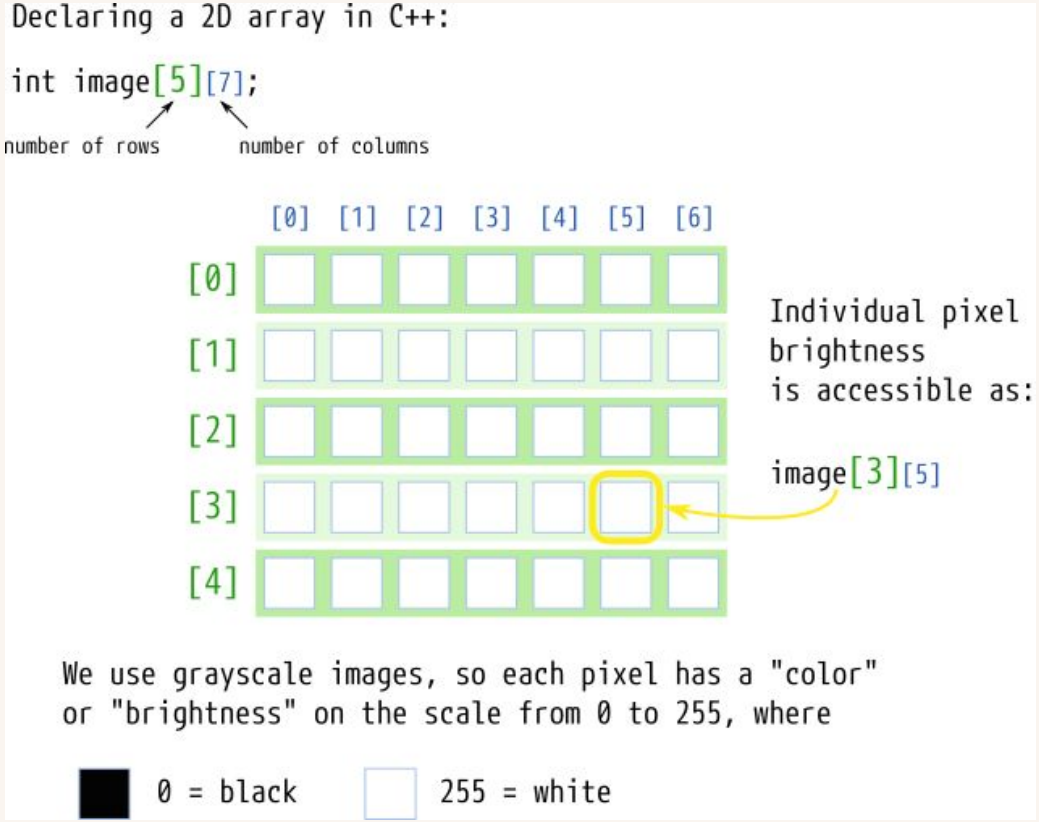
Lab 8: https://maryash.github.io/135/labs/lab_08.html

Ryan Vaz

PGM formatted files

In this lab, we will be working with image files that are stored in the PGM format. The file contains pixel data that can be represented in a 2D array. A sample image is provided in the lab.

If you want to view these images, I'd suggest installing the `PBM/PPM/PGM Viewer for Visual Studio Code` extension so you can view the images without the need for the eye of gnome (eog) utility.



Since we are working with greyscale, each index only has 1 value (unlike 127 where we need RGB at each index). 0 is black, and 255 is white.

Since we need to use a 2D array, we'll be traversing the rows and columns using a nested for loop where the outer loop represents the rows, and inner loop represents the columns, thus to access a individual pixel: image[row][col]

Reading and Writing Pictures

void readImage(int image[MAX_H][MAX_W], int &height, int &width);
This function reads the pixel colors into the array, and updates the variables height and width with the actual dimensions of the loaded image.

void writeImage(int image[MAX_H][MAX_W], int height, int width);
This function saves the given image array of given height and width into an image file that will be called `outImage.pgm`. All the pixels in the array MUST be between 0 and 255(inclusive).

The provided main function creates an array `img`, reads the picture from the `inImage.pgm` into this array, copies this array to a second array of the same dimensions, and writes this second array into the output file `outImage.pgm`.

Things to consider

Don’t modify the two provided functions. Only make changes to the main function or write your own functions that can be used in the main function.

If you look at the main function provided, you will see that instead of modifying the array of the image we are reading, we are creating a new array. For this lab, we will use that new array and **never modify the array of the original image**.

The size of the array in provided main function is 512 X 512 which is significantly larger than the image that we are using. That is totally fine as the function will keep track of the dimensions. **If you are using your own image file, make sure that the height and the width of the image is less than 512.**

```
int main() {

    int img[MAX_H][MAX_W];
    int h, w;

    readImage(img, h, w);
    int out[MAX_H][MAX_W];

    for(int row = 0; row < h; row++) {
        for(int col = 0; col < w; col++) {
            out[row][col] = img[row][col];
        }
    }
    writeImage(out, h, w);

}
```

Declare a 2D array with the MAX_H, and MAX_W being the size of the rows and columns

Send the array, h, and w, into the function readImage by reference, and update their values based on `inImage.pgm`

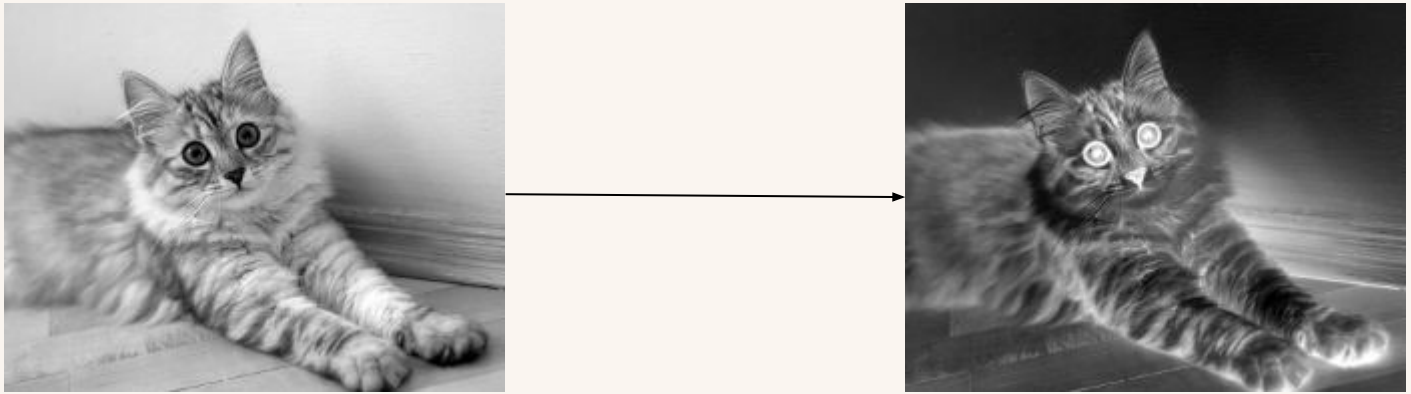
Make a new array called out with the same size params as img

Set the value of the new image equal to the value of the old image for all [row, col]

Write the new image to a file `outImage.pgm`

Task A

Our program needs to invert all the pixels of the input image:



Inverting the pixels

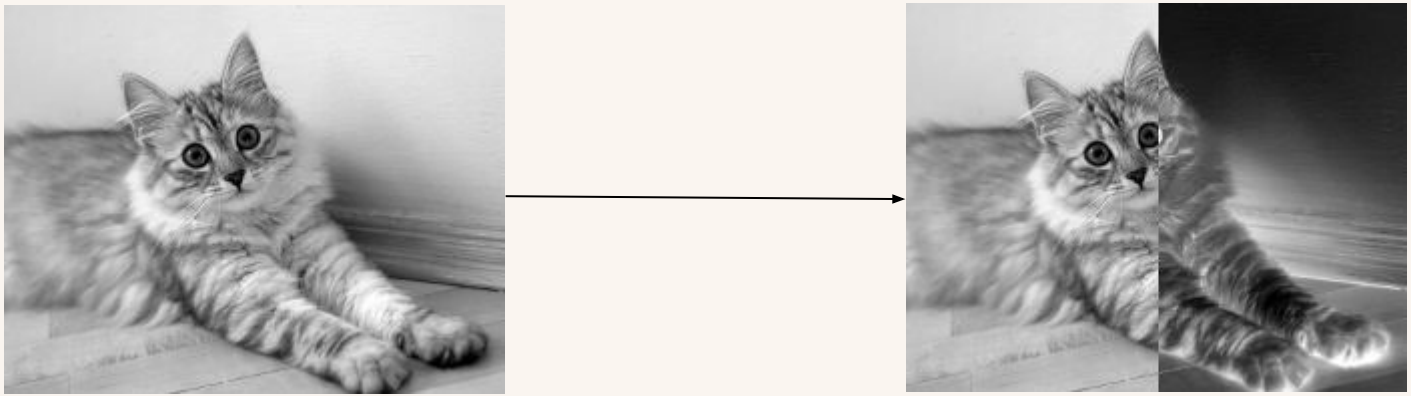
We need to find a way to invert all the pixels. Let's look for a pattern:

$0 \rightarrow 255$	$255 - 0 = 255$
$1 \rightarrow 254$	$255 - 1 = 254$
$2 \rightarrow 253$	$255 - 2 = 253$
...	...
$255 \rightarrow 0$	$255 - 255 = 0$

Therefore, 255 - pixel = inverted pixel
Within our nested for loop, the inverted pixel of `image[row][col]` would be: `255 - image[row][col]`

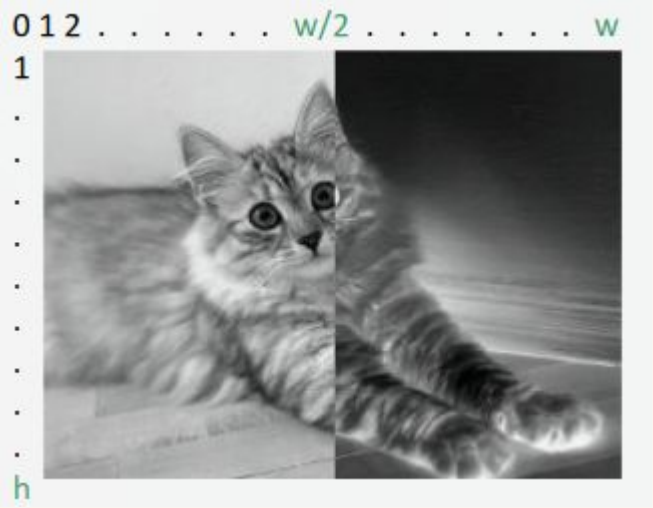
Task B

We will invert the pixels in the right half of the image.



Isolating the right half of the image

Let's identify what the right half of the image is based on the height and width.



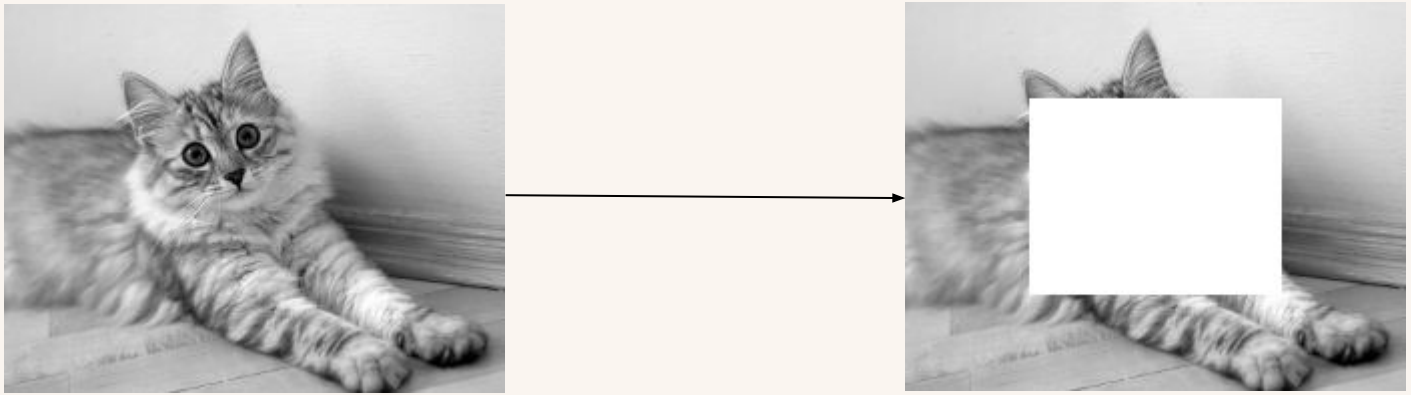
h : maximum height
w : maximum width
w/2 : half of the maximum width

The right half of the image occurs when `col` is greater than half of the maximum width.

Therefore we can use an if statement to check if `col>=w/2` and invert the pixels if that is the case. Otherwise set the pixel equal to the original pixel.

Task C

For this task, our program will produce a rectangular box in the middle of the image. The box is 50% by 50% of the original image height and width:



Isolating the center of the image

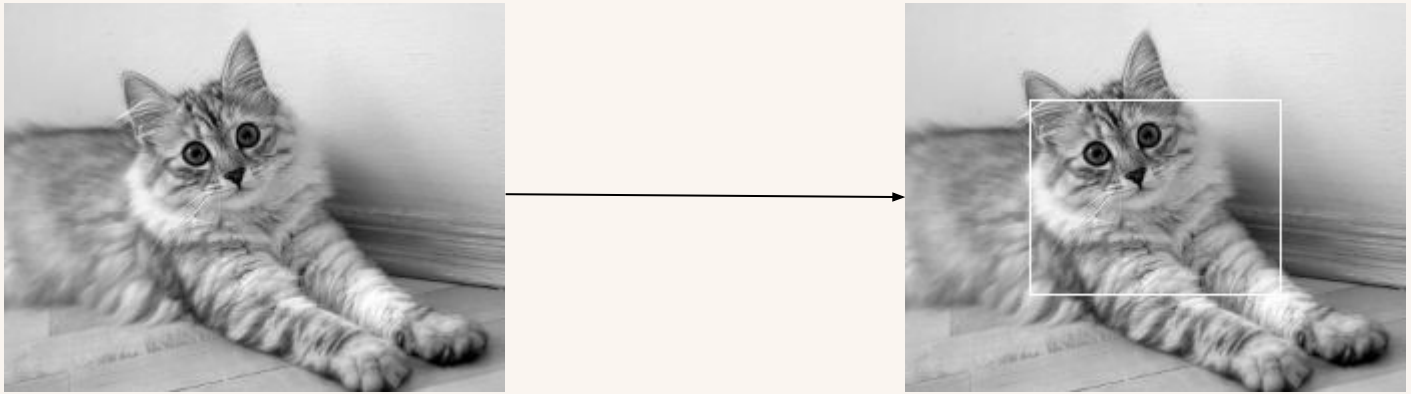
Let's identify what the 50% in the center of the image is based on the height and width.

h : maximum height
w : maximum width
w/4 : first quarter of the width
h/4 : first quarter of the height
(w/4)*3 : third quarter of the width
(h/4)*3 : third quarter of the height

Pixels that are greater than the first quarter of the height and width, but smaller than the third quarter of the height and width are set to white.

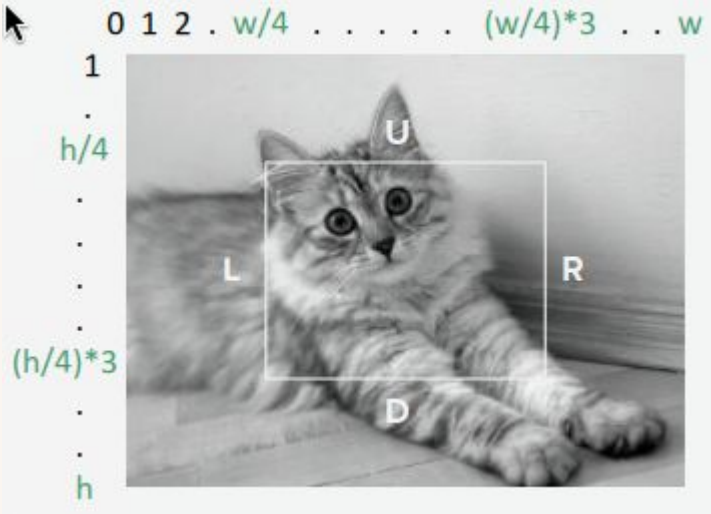
Task D

Produce a rectangular frame in the middle of the image:



Isolating the center of the image

Isolate the lines individually using an if-statement and set the pixels to white.



U: row==h/4 && (col>=w/4 && col<=(w/4)*3)
D: row==(h/4)*3 && (col>=w/4 && col<=(w/4)*3)
L: col==(w/4) && (row>=h/4 && row<=(h/4)*3)
R: col==(w/4)*3 && (row>=h/4 && row<=(h/4)*3)

Task E

Scale the image upto 200% of its original size

Small Pixels vs Large Pixels

Make the assumption that the image even when doubled in size, the image will be smaller than the array size limit of 512.

Since the output image is 2x the size of the original image, output[row*2][col*2] = original[row][col]

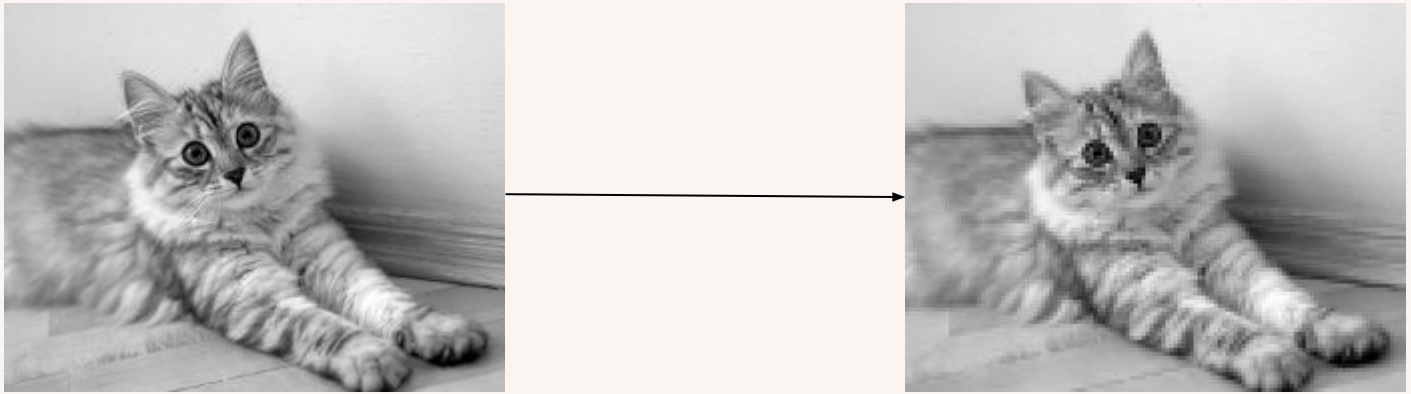
Let's see how the rows and columns are related:
original[0][0] = original[0][0]
original[0][0] = original[0][1]
original[0][0] = original[1][0]
original[0][0] = original[1][1]

11 22 → 11 11 22 22
33 44 → 11 11 22 22
→ 33 33 44 44
33 33 44 44

Based on this, we can tell:
output [row*2][col*2] = original[row][col]
output [row*2][(col*2)+1] = original[row][col]
output [(row*2)+1][col*2] = original[row][col]
output [(row*2)+1][(col*2)+1] = original[row][col]

Task F

Write a program to pixelate the input image:



Pixelating the image

Pixelating an image means replacing every four adjacent pixels to the average of the pixels.

The adjacent pixels are:
A - img[row][col]
B - img[row+1][col]
C - img[row][col+1]
D - img[row+1][col+1]

Average = (A+B+C+D)/4
Set ALL the adjacent pixels of the output image to the average

10 40 12 32 → 20 20 16 16
10 20 10 10 → 20 20 16 16
15 45 20 10 → 22 22 15 15
30 00 18 12 → 22 22 15 15

(10 + 40 + 10 + 20)/4 = 80/4 = 20
(12 + 32 + 10 + 10)/4 = 64/4 = 16
(15 + 45 + 30 + 0)/4 = 90/4 = 22(.5)
(20 + 10 + 18 + 12)/4 = 60/4 = 15